



Indian Institute of Technology Guwahati

BSc. (Hons.) Data Science & Artificial Intelligence
Online Degree Programme

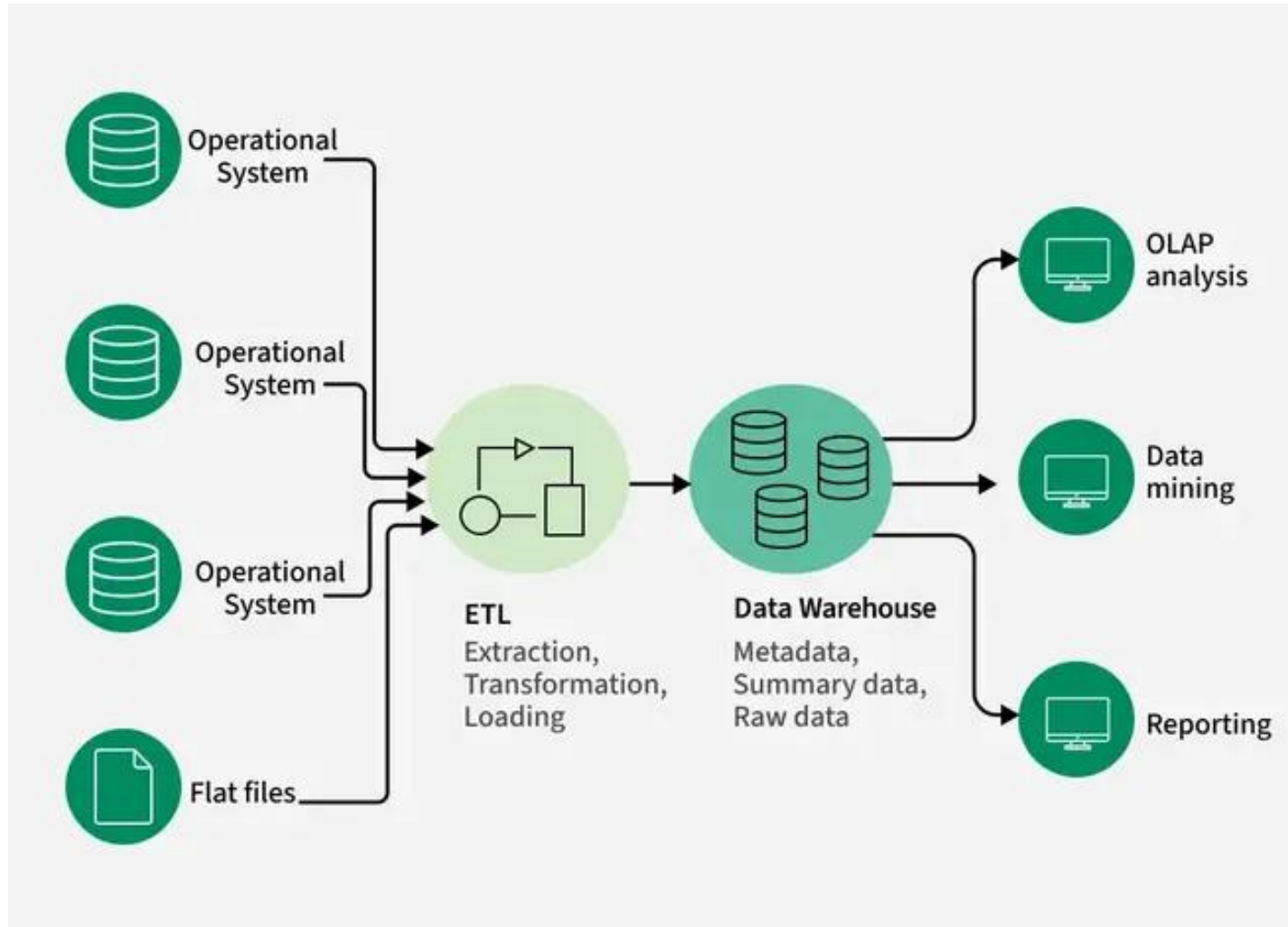
INTERNSHIP/TERM PROJECT - 1

Medallion Data Warehouse Pipeline

Student Name :	Prahlad Chilwal
Student ID :	23035010546
Email ID :	c.prahlad@iitg.ac.in
Programme :	Online B.Sc. (Honours)
Course Code :	DA 377
Course Name :	Internship/Term Project - 1
Trimester :	VII

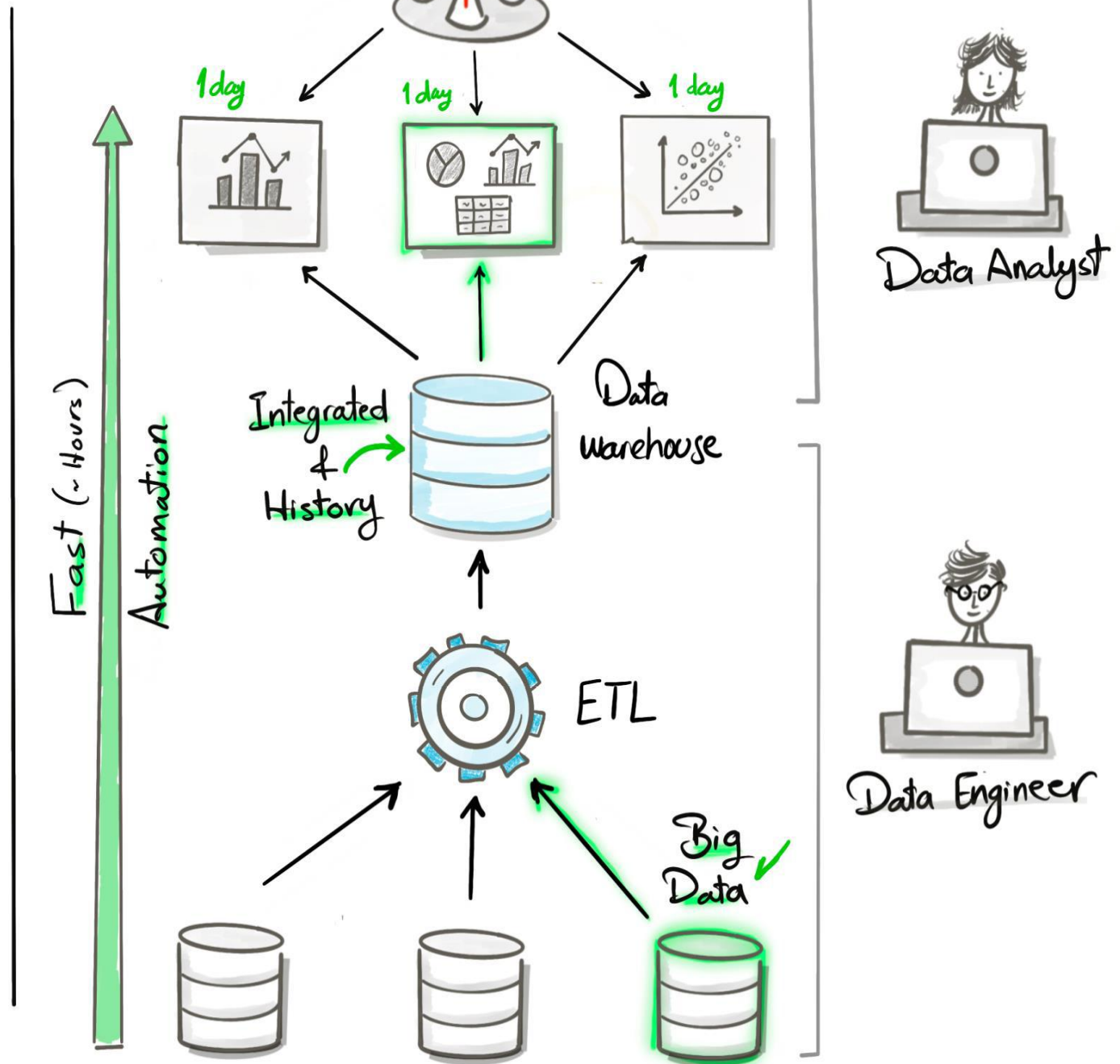
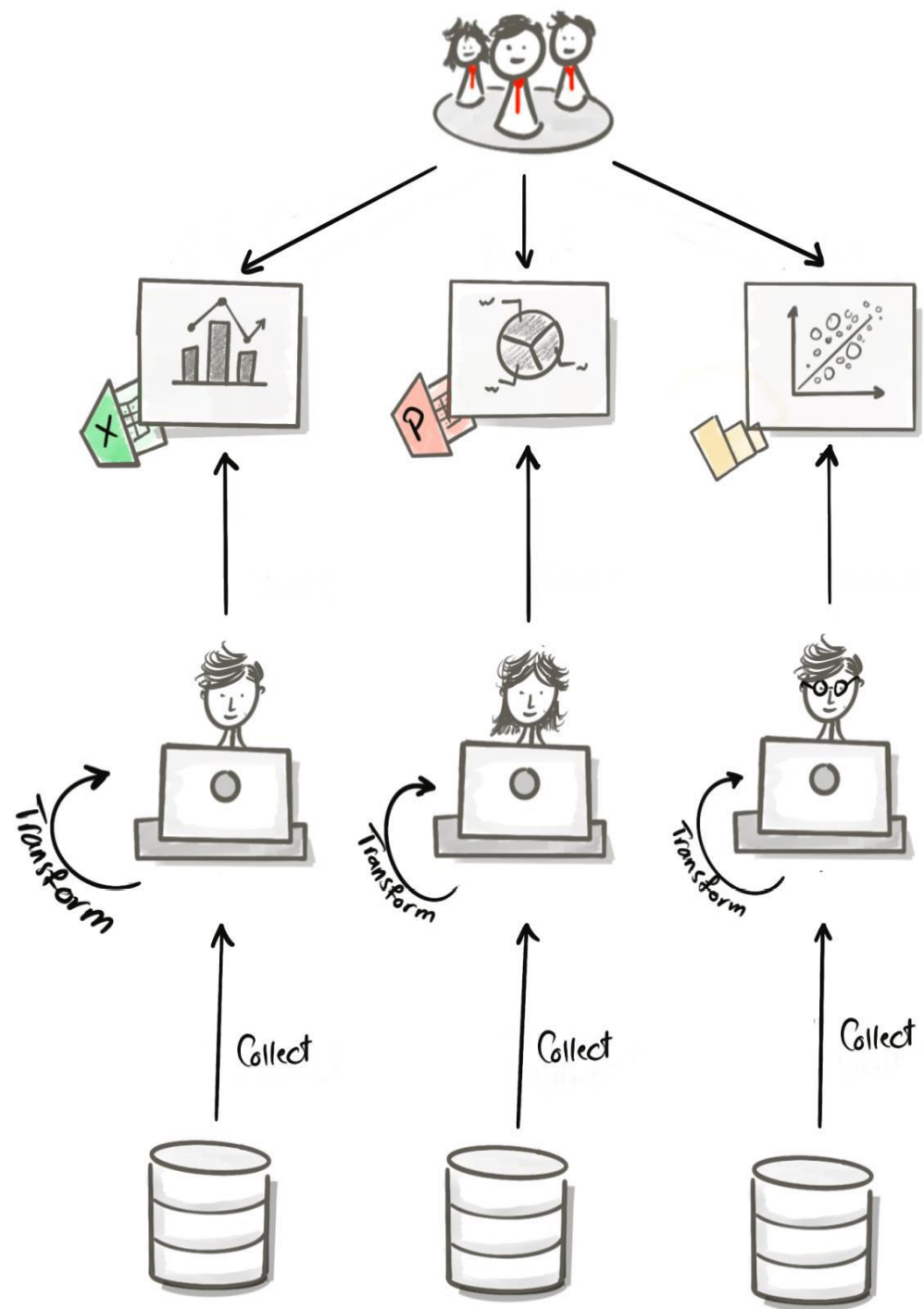
What Is Data Warehouse?

Data warehousing is the process of collecting, integrating, storing and managing data from multiple sources in a central repository. It enables organizations to organize large volumes of current and historical data for efficient querying, analysis and reporting.



Need For Data Warehouse

- **Handling Large Data Volumes:** Traditional databases store limited data (MBs to GBs), while data warehouses are built to handle huge datasets (up to TBs), making it easier to store and analyze long-term historical data.
- **Enhanced Analytics:** Databases handle transactions; data warehouses are optimized for complex analysis and historical insights.
- **Centralized Data Storage:** A data warehouse combines data from multiple sources, giving a single, unified view for better decision-making.
- **Trend Analysis:** By storing historical data, a data warehouse allows businesses to analyze trends over time, enabling them to make strategic decisions based on past performance and predict future outcomes.
- **Business Intelligence Support:** Data warehouses work with BI tools to give quick access to insights, helping in data-driven decisions and improving efficiency.



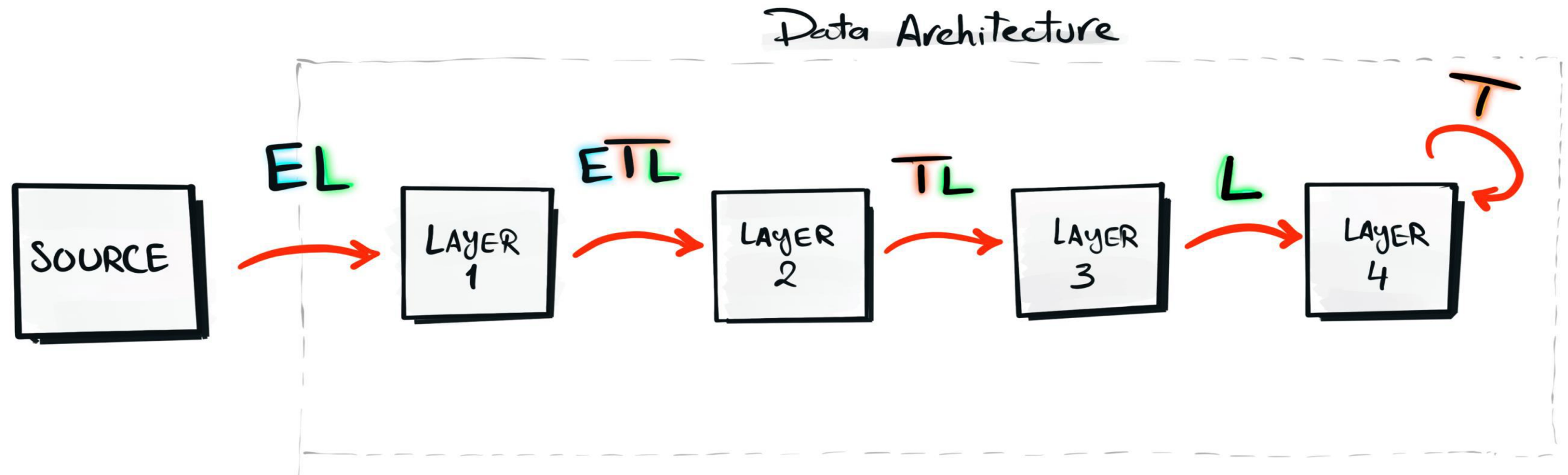
Components of Data Warehouse

- **Data Sources:** These are the various operational systems, databases and external data feeds that provide raw data to be stored in the warehouse.
- **ETL (Extract, Transform, Load) Process:** The ETL process is responsible for extracting data from different sources, transforming it into a suitable format and loading it into the data warehouse.
- **Data Warehouse Database:** This is the central repository where cleaned and transformed data is stored. It is typically organized in a multidimensional format for efficient querying and reporting.
- **Metadata:** Metadata describes the structure, source and usage of data within the warehouse, making it easier for users and systems to understand and work with the data.
- **Data Marts:** These are smaller, more focused data repositories derived from the data warehouse, designed to meet the needs of specific business departments or functions.
- **OLAP (Online Analytical Processing) Tools:** [OLAP tools](#) allow users to analyze data in multiple dimensions, providing deeper insights and supporting complex analytical queries.
- **End-User Access Tools:** These are reporting and analysis tools, such as dashboards or [Business Intelligence \(BI\) tools](#), that enable business users to query the data warehouse and generate reports.

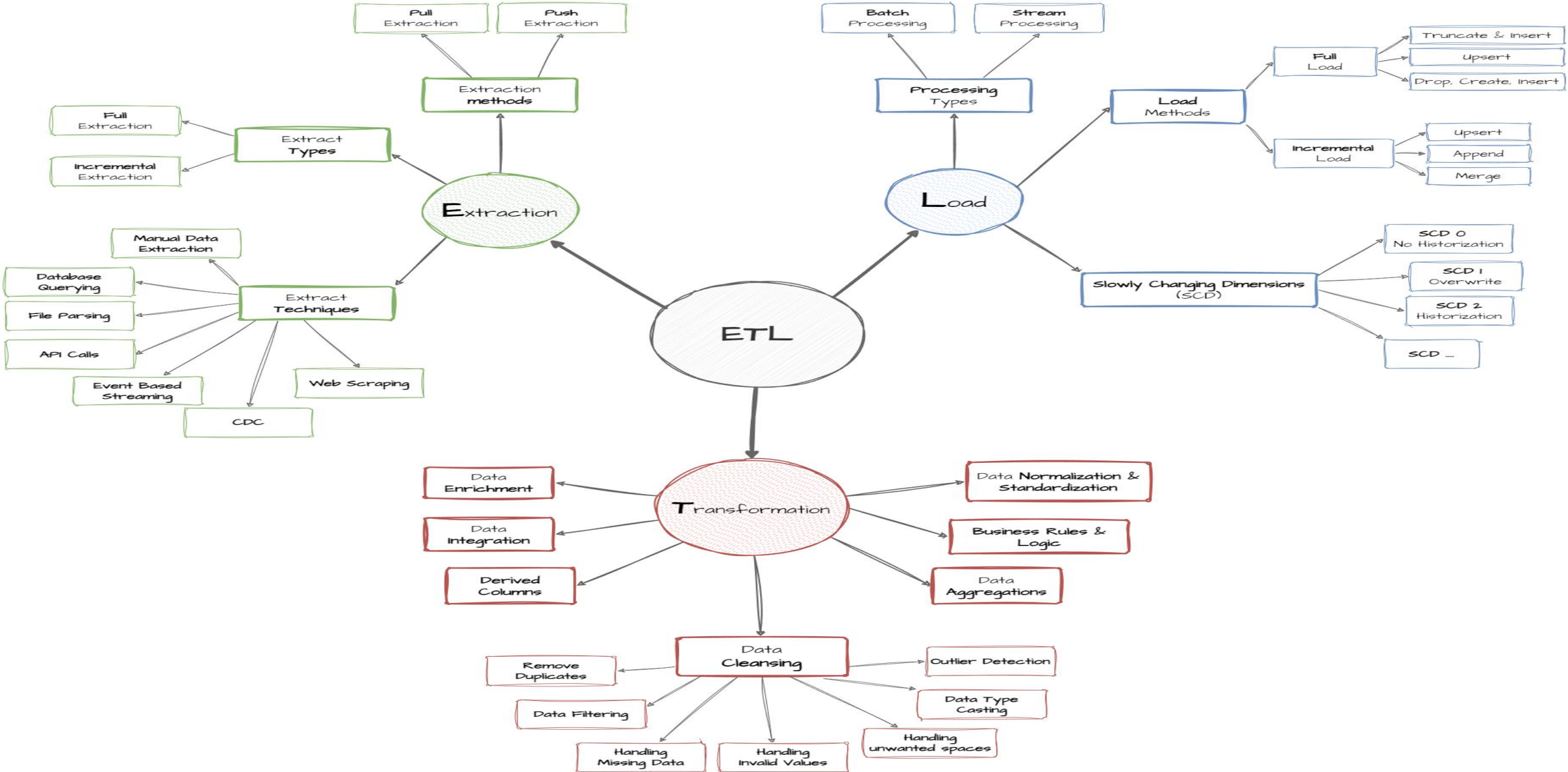
What Is ETL?

ETL (Extract, Transform, Load) is a key process in data warehousing that prepares data for analysis. It involves:

- Extracting data from multiple sources
- Transforming it into a consistent format
- Loading it into a central data warehouse or data lake



ETL METHODS



PROJECT INITIALIZATION

PROJECT REQUIREMENTS

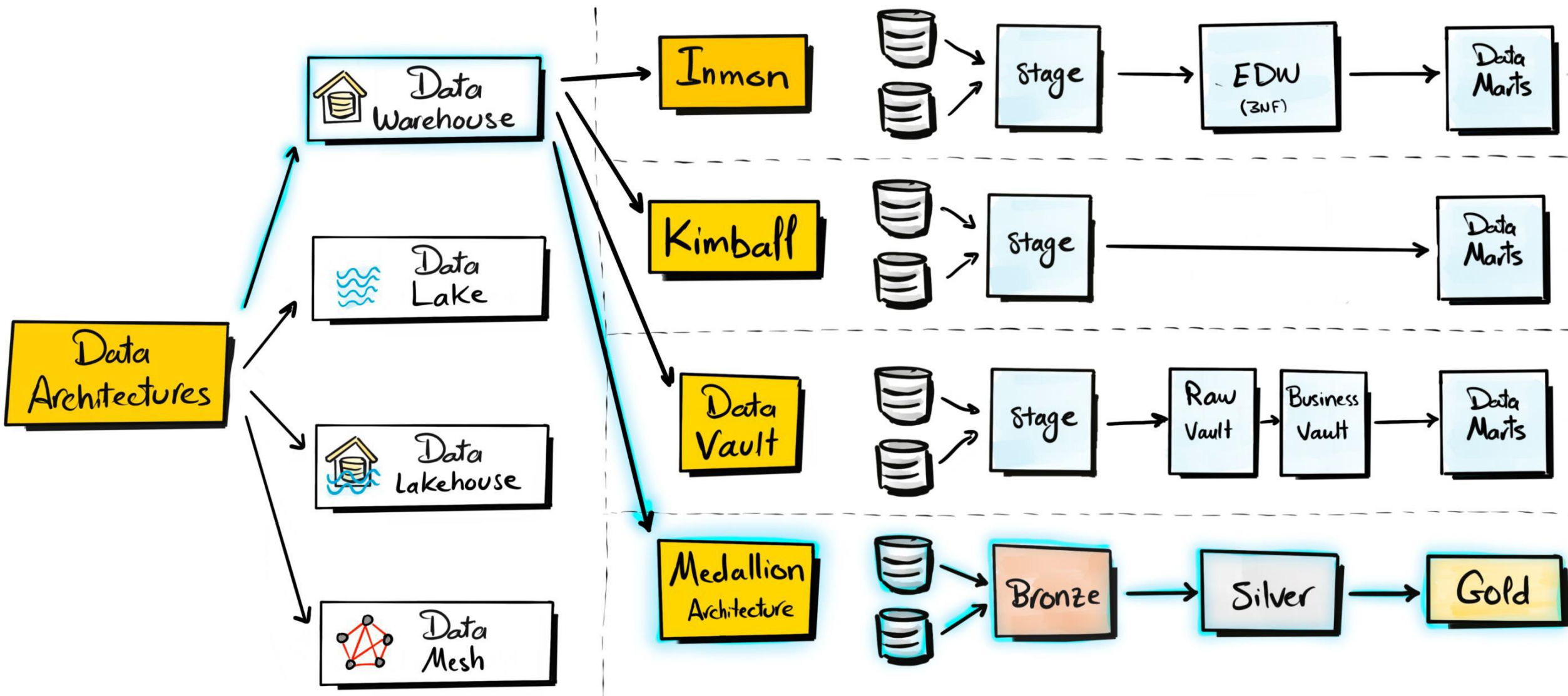
Building the Medallion Data Warehouse (Olist E-commerce) :-

Objective :

Develop a modern data warehouse using the **Medallion Architecture** to process Olist E-commerce data, transforming raw marketplace records into curated datasets for advanced business intelligence and trend analysis.

Specifications :

- **Data Sources:** Ingest raw Olist E-commerce data (customers, orders, products, payments, and reviews) provided as multiple relational CSV files.
- **Data Quality (Bronze to Silver):** Implement a "Silver" layer to cleanse null values, standardize date formats, and resolve schema inconsistencies found in the raw landing zone.
- **Integration (Silver to Gold):** Combine disparate tables into a high-performance "Gold" layer, utilizing a **Star Schema** (Fact and Dimension tables) optimized for sales performance and logistics queries.
- **Scope:** Focus on the full Olist dataset lifecycle; while historization (SCD Type 2) is optional, the pipeline must ensure data integrity across the three architectural layers.
- **Documentation:** Provide a comprehensive data dictionary and lineage map tracing data from the original Olist CSVs through the Silver transformation to the final Gold business entities.



Note : We use Data warehouse Medallion Architecture.

DESIGNING LAYERS OF DATA WAREHOUSE



Bronze Layer



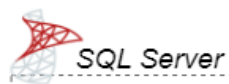
Silver Layer



Gold Layer

	Bronze Layer	Silver Layer	Gold Layer
Definition	Raw, unprocessed data as-is from sources	Clean & standardized data	Business-Ready data
Objective	Traceability & Debugging	(Intermediate Layer) Prepare Data for Analysis	Provide data to be consumed for reporting & Analytics
Object Type	Tables	Tables	Views
Load Method	Full Load (Truncate & Insert)	Full Load (Truncate & Insert)	None
Data Transformation	None (as-is)	- Data Cleaning - Data Standardization - Data Normalization - Derived Columns - Data Enrichment	- Data Integration - Data Aggregation - Business Logic & Rules
Data Modeling	None (as-is)	None (as-is)	- Star Schema - Aggregated Objects - Flat Tables
Target Audience	- Data Engineers	- Data Analysts - Data Engineers	- Data Analysts - Business Users

DESIGNING DATA ARCHITECTURE



Sources



CRM



ERP

Object Type: CSV Files

Interface: Files in Folders



Bronze Layer

Stored Procedure



Raw Data

Object Type: Tables

Load:

- Batch Processing
- Full Load
- Truncate & Insert

No Transformations

Data Model: None (as-is)



Data Warehouse

Silver Layer

Stored Procedure



Cleaned, Standardized Data

Object Type: Tables

Load:

- Batch Processing
- Full Load
- Truncate & Insert

Transformations:

- Data Cleansing
- Data Standardization
- Data Normalization
- Derived Columns
- Data Enrichment

Data Model: None (as-is)



Gold Layer



Business-Ready Data

Object Type: Views

No Load

Transformations:

- Data Integrations
- Aggregations
- Business Logics

Data Model:

- Star Schema
- Flat Table
- Aggregated Table



Consume



BI & Reporting



Ad-Hoc SQL Queries



Machine Learning

CREATING DATABASE 'DataWarehouse'

```
USE master;
GO

-- Drop and recreate the 'DataWarehouse' database
IF EXISTS (SELECT 1 FROM sys.databases WHERE name = 'DataWarehouse')
BEGIN
    ALTER DATABASE DataWarehouse SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE DataWarehouse;
END;
GO

-- Create the 'DataWarehouse' database
CREATE DATABASE DataWarehouse;
GO

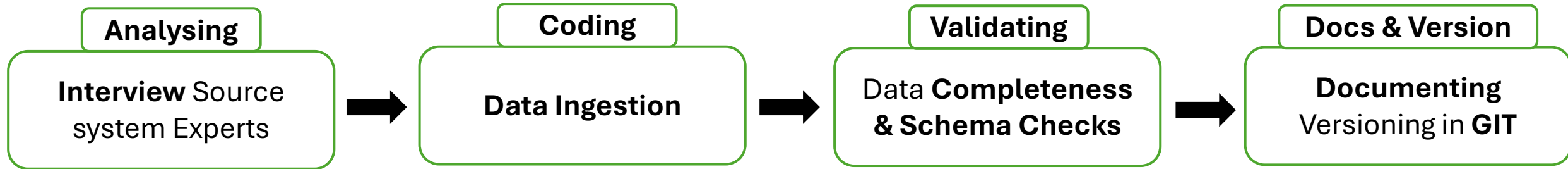
USE DataWarehouse;
GO

-- Create Schemas
CREATE SCHEMA bronze;
GO

CREATE SCHEMA silver;
GO

CREATE SCHEMA gold;
GO
```

BUILDING BRONZE LAYER



Interview Source system Experts

Business Context & Ownership

Who **owns** the **data**?

What **Business Process** it supports?

System & Data **documentation**

Data **Model** & Data **Catalog**

Architecture & Technology Stack

How is data **stored**?
(SQL Server, Oracle, AWS, Azure, ...)

What are the **integration capabilities**?
(API, Kafka, File Extract, Direct DB, ...)

Extract & Load

Incremental vs. **Full Loads** ?

Data **Scope** & **Historical** Needs

What is the expected **size** of the **extracts**?

Are there any data **volume limitations**?

How to **avoid impacting** the source
system's **performance**?

authentication and **authorization**
(tokens, SSH keys, VPN, IP whitelisting, ...)

BUILD BRONZE LAYER CREATE DDL FOR TABLES

LOAD_SCRIPT_...Prahla (52) | SQLQuery2.sql...Prahla (71) | SQLQuery1.sql...Prahla (65) | DDL_BRONZE.s...rahlad (51) |

```
1  /*
2  =====
3  DDL Script: Create Bronze Tables
4  =====
5  Script Purpose:
6      This script creates tables in the 'bronze' schema, dropping existing tables
7      if they already exist.
8      Run this script to re-define the DDL structure of 'bronze' Tables
9  =====
10 */
11
12 -- 1. Customers Table
13 IF OBJECT_ID('bronze.crm_customers', 'U') IS NOT NULL
14     DROP TABLE bronze.crm_customers;
15 GO
16 CREATE TABLE bronze.crm_customers(
17     customer_id          NVARCHAR(100),
18     customer_unique_id   NVARCHAR(100),
19     customer_zip_code_prefix INT,
20     customer_city         NVARCHAR(100),
21     customer_state       NVARCHAR(100)
22 );
23 GO
24
25 -- 2. Geolocation Table
26 IF OBJECT_ID('bronze.crm_geolocation', 'U') IS NOT NULL
27     DROP TABLE bronze.crm_geolocation;
28 GO
29 CREATE TABLE bronze.crm_geolocation(
30     geolocation_zip_code_prefix INT,
31     geolocation_lat             DECIMAL(12, 9),
32     geolocation_lng             DECIMAL(12, 9),
33     geolocation_city            NVARCHAR(100),
34     geolocation_state           NVARCHAR(100)
35 );
36 GO
37
38 -- 3. Order Reviews Table
39 IF OBJECT_ID('bronze.crm_order_reviews', 'U') IS NOT NULL
40     DROP TABLE bronze.crm_order_reviews;
41 GO
42 CREATE TABLE bronze.crm_order_reviews(
43     review_id          NVARCHAR(100),
44     order_id           NVARCHAR(100),
45     review_score        INT,
46     review_comment_title NVARCHAR(255),
47     review_comment_message NVARCHAR(MAX), -- Increased to handle long Portuguese reviews
48     review_creation_date DATETIME,
49     review_answers_timestamp DATETIME
50 );
51 GO
52
53 -- 4. Order Items Table
54 IF OBJECT_ID('bronze.erp_order_items', 'U') IS NOT NULL
55     DROP TABLE bronze.erp_order_items;
56 GO
57 CREATE TABLE bronze.erp_order_items(
58     order_id          NVARCHAR(100),
59     order_item_id     INT,
```


BUILD BRONZE LAYER CREATE SQL LOAD SCRIPTS

LOAD_SCRIPT_...rahlad (52) SQLQuery2.sql...Prahla (71) SQLQuery1.sql...Prahla (65) DDL_BRONZE.s...Prahla (51)

```
1  /*
2  =====
3  Stored Procedure: Load Bronze Layer (Source -> Bronze)
4  =====
5  Script Purpose:
6  This stored procedure loads data into the 'bronze' schema from external CSV files.
7  It performs the following actions:
8  - Truncates the bronze tables before loading data.
9  - Uses the 'BULK INSERT' command to load data from csv Files to bronze tables.
10
11 Parameters:
12 None.
13 This stored procedure does not accept any parameters or return any values.
14
15 Usage Example:
16 EXEC bronze.load_bronze;
17 =====
18 */
19
20 CREATE OR ALTER PROCEDURE bronze.usp_BulkLoadOlist
21 AS
22 BEGIN
23     DECLARE @start_time DATETIME, @end_time DATETIME, @batch_start_time DATETIME, @batch_end_time DATETIME;
24     BEGIN TRY
25         SET @batch_start_time = GETDATE();
26         PRINT '=====';
27         PRINT 'STARTING OLIST BRONZE LAYER BULK LOAD';
28         PRINT '=====';
29
30         PRINT '-----';
31         PRINT 'LOADING CRM TABLES';
32         PRINT '-----';
33         PRINT ''; -- Space after header
34
35         -- 1. Customers
36         SET @start_time = GETDATE();
37         PRINT '>> TRUNCATING TABLE: bronze.crm_customers';
38         TRUNCATE TABLE bronze.crm_customers;
39
40         PRINT '>> INSERTING DATA INTO: bronze.crm_customers';
41
42         BULK INSERT bronze.crm_customers
43         FROM 'C:\OlistData\Olist Brazilian E-Commerce dataset\Source System 1 CRM (Front-Office)\customers.csv'
44         WITH (FORMAT = 'CSV', FIRSTROW = 2, FIELDTERMINATOR = ',', ROWTERMINATOR = '0x0a', CODEPAGE = '65001', TABLOCK);
45         PRINT '>> Customers loaded.';
46         SET @end_time = GETDATE();
47         PRINT '>> Load Duration: ' + CAST(DATEDIFF(second, @start_time, @end_time) AS NVARCHAR) + ' seconds';
48         PRINT '>> -----';
49         PRINT ''; -- Space after table 1
50
51         -- 2. Geolocation
52         SET @start_time = GETDATE();
53         PRINT '>> TRUNCATING TABLE: bronze.crm_geolocation';
54         TRUNCATE TABLE bronze.crm_geolocation;
55
56         PRINT '>> INSERTING DATA INTO: bronze.crm_geolocation';
57
58         BULK INSERT bronze.crm_geolocation
59         FROM 'C:\OlistData\Olist Brazilian E-Commerce dataset\Source System 1 CRM (Front-Office)\geolocation.csv'
```

63 % No issues found

Connected. (1/1)

Ln: 19 Ch: 1 MIXED LF

LAPTOP-VGO36MDC\SQLEXPRESS ... LAPTOP-VGO36MDC\Prahla... DataWarehouse 00:00:00 0 rows

Connect ▾ 

- FileTables
- External Tables
- Graph Tables
- bronze.crm_customers
- bronze.crm_geolocation
- bronze.crm_order_reviews
- bronze.erp_order_items
- bronze.erp_order_payments
- bronze.erp_orders
- bronze.erp_products
- bronze.erp_sellers
- bronze.product_category_name_translation
- dbo.bronze_product_category_name_translation
- silver.crm_customers
- silver.crm_geolocation
- silver.crm_order_reviews
- silver.erp_order_items
- silver.erp_order_payments
- silver.erp_orders
- silver.erp_products
- silver.erp_sellers
- silver.product_category_name_translation
- Views
- External Resources
- Synonyms
- Programmability
 - Stored Procedures
 - System Stored Procedures
 - bronze.usp_BulkLoadOlist**
 - silver.usp_BulkLoadSilver
 - Functions
 - Database Triggers
 - Assemblies
 - Types
 - Rules
 - Defaults

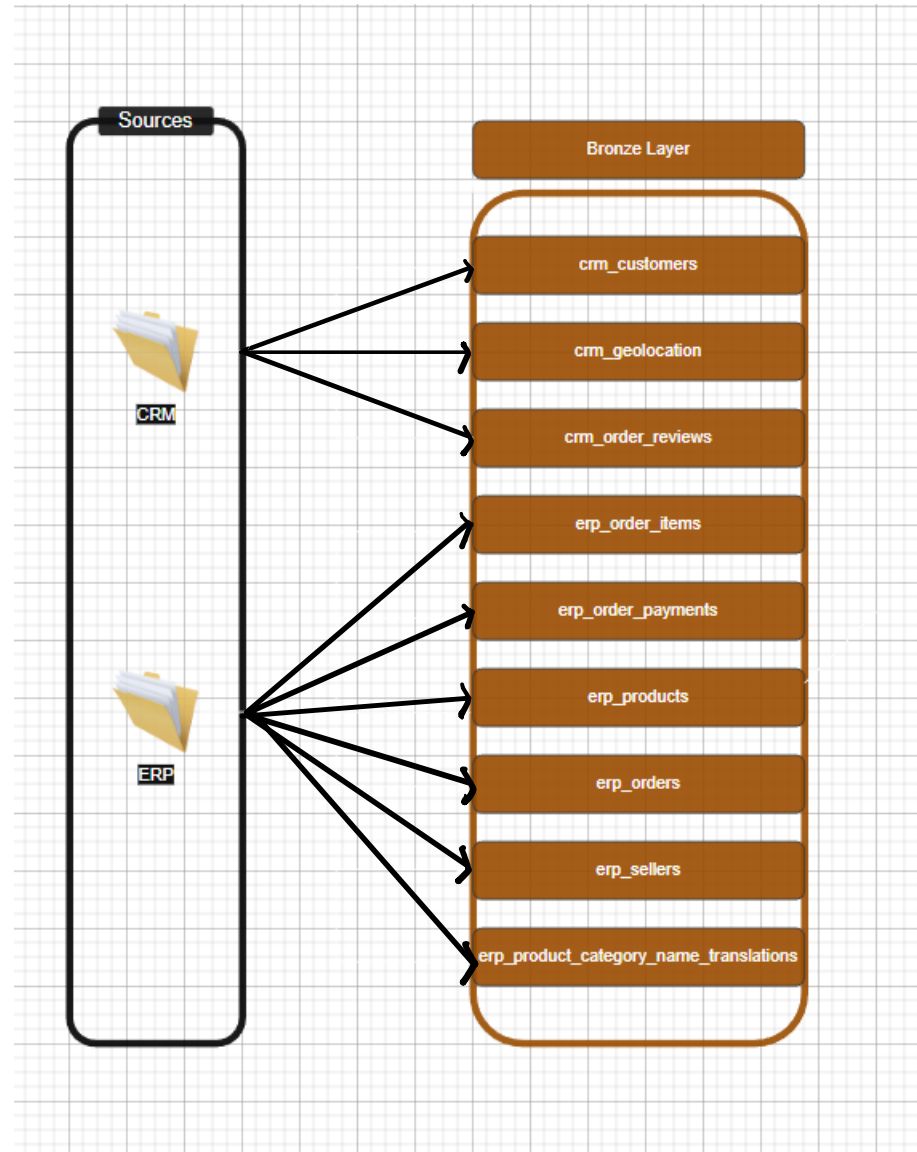
```
1  /*
2  =====
3  Stored Procedure: Load Bronze Layer (Source -> Bronze)
4  =====
5  Script Purpose:
6      This stored procedure loads data into the 'bronze' schema from external CSV files.
7      It performs the following actions:
8      - Truncates the bronze tables before loading data.
9      - Uses the 'BULK INSERT' command to load data from csv Files to bronze tables.
10
11 Parameters:
12     None.
13     This stored procedure does not accept any parameters or return any values.
14
15 Usage Example:
16     EXEC bronze.load_bronze;
17 =====
18 */
19
20 CREATE OR ALTER PROCEDURE bronze.usp_BulkLoadOlist
21 AS
22 BEGIN
23     DECLARE @start_time DATETIME, @end_time DATETIME, @batch_start_time DATETIME, @batch_end_time DATETIME;
24     BEGIN TRY
25         SET @batch_start_time = GETDATE();
26         PRINT '=====';
27         PRINT 'STARTING OLIST BRONZE LAYER BULK LOAD';
28         PRINT '=====';
29
30         PRINT '-----';
31         PRINT 'LOADING CRM TABLES';
32         PRINT '-----';
33         PRINT ''; -- Space after header
34
35         -- 1. Customers
36         SET @start_time = GETDATE();
37         PRINT '>> TRUNCATING TABLE: bronze.crm_customers';
38         TRUNCATE TABLE bronze.crm_customers;
39
40         PRINT '>> INSERTING DATA INTO: bronze.crm_customers';
41
42         BULK INSERT bronze.crm_customers
43         FROM 'C:\OlistData\Olist Brazilian E-Commerce dataset\Source System 1 CRM (Front-Office)\customers.csv'
44         WITH (FORMAT = 'CSV', FIRSTROW = 2, FIELDTERMINATOR = ',', ROWTERMINATOR = '0x0a', CODEPAGE = '65001', TABLOCK);
45         PRINT '>> Customers loaded.';
46         SET @end_time = GETDATE();
47         PRINT '>> Load Duration: ' + CAST(DATEDIFF(second, @start_time, @end_time) AS NVARCHAR) + ' seconds';
48         PRINT '>> -----';
49         PRINT ''; -- Space after table 1
50
51         -- 2. Geolocation
52         SET @start_time = GETDATE();
53         PRINT '>> TRUNCATING TABLE: bronze.crm_geolocation';
54         TRUNCATE TABLE bronze.crm_geolocation;
55
56         PRINT '>> INSERTING DATA INTO: bronze.crm_geolocation';
57
58         BULK INSERT bronze.crm_geolocation
59         FROM 'C:\OlistData\Olist Brazilian E-Commerce dataset\Source System 1 CRM (Front-Office)\geolocation.csv'
```

63 %  No issues found

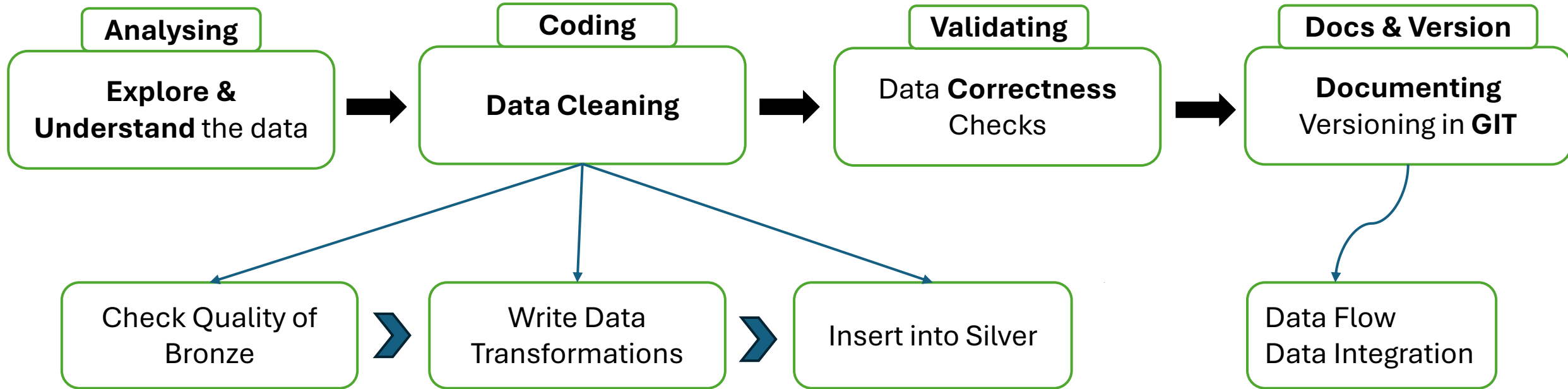
Ln: 19 Ch: 1 MIXED LF

Connected. (1/1) | LAPTOP-VGO36MDC\SQLEXPRESS... | LAPTOP-VGO36MDC\Prahla... | DataWarehouse | 00:00:00 | 0 rows

DATA FLOW DIAGRAM



BUILDING SILVER LAYER



BUILD SILVER LAYER

CREATE DDL FOR TABLES

DDL_SILVER.s...rahlad (51)

```
1  /*
2  =====
3  DDL Script: Create silver Tables
4  =====
5  Script Purpose:
6      This script creates tables in the 'silver' schema, dropping existing tables
7      if they already exist.
8      Run this script to re-define the DDL structure of 'silver' Tables
9  =====
10 */
11
12 -- 1. Customers Table
13 IF OBJECT_ID('silver.crm_customers', 'U') IS NOT NULL
14     DROP TABLE silver.crm_customers;
15 GO
16 CREATE TABLE silver.crm_customers(
17     customer_id          NVARCHAR(100),
18     customer_unique_id   NVARCHAR(100),
19     customer_zip_code_prefix INT,
20     customer_city        NVARCHAR(100),
21     customer_state       NVARCHAR(100),
22     dwh_create_date      DATETIME2 DEFAULT GETDATE()
23 );
24 GO
25
26 -- 2. Geolocation Table
27 IF OBJECT_ID('silver.crm_geolocation', 'U') IS NOT NULL
28     DROP TABLE silver.crm_geolocation;
29 GO
30 CREATE TABLE silver.crm_geolocation(
31     geolocation_zip_code_prefix INT,
32     geolocation_lat            DECIMAL(12, 9),
33     geolocation_lng            DECIMAL(12, 9),
34     geolocation_city           NVARCHAR(100),
35     geolocation_state          NVARCHAR(100),
36     dwh_create_date            DATETIME2 DEFAULT GETDATE()
37 );
38 GO
39
40 -- 3. Order Reviews Table
41 IF OBJECT_ID('silver.crm_order_reviews', 'U') IS NOT NULL
42     DROP TABLE silver.crm_order_reviews;
43 GO
44 CREATE TABLE silver.crm_order_reviews(
45     review_id          NVARCHAR(100),
46     order_id           NVARCHAR(100),
47     review_score        INT,
48     review_comment_title NVARCHAR(255),
49     review_comment_message NVARCHAR(MAX), -- Increased to handle long Portuguese reviews
50     review_creation_date DATETIME,
51     review_answers_timestamp DATETIME,
52     dwh_create_date      DATETIME2 DEFAULT GETDATE()
53 );
54 GO
```

- FileTables
- External Tables
- Graph Tables
- bronze.crm_customers
- bronze.crm_geolocation
- bronze.crm_order_reviews
- bronze.erp_order_items
- bronze.erp_order_payments
- bronze.erp_orders
- bronze.erp_products
- bronze.erp_sellers
- bronze.product_category_name_translation
- dbo.bronze_product_category_name_translation
- silver.crm_customers**
- silver.crm_geolocation**
- silver.crm_order_reviews**
- silver.erp_order_items**
- silver.erp_order_payments**
- silver.erp_orders**
- silver.erp_products**
- silver.erp_sellers**
- silver.product_category_name_translation**
- Views
- External Resources
- Synonyms
- Programmability
 - Stored Procedures
 - System Stored Procedures
 - bronze.usp_BulkLoadOlist
 - silver.usp_BulkLoadSilver
- Functions
- Database Triggers
- Assemblies
- Types
- Rules
- Defaults

```
1  s/*
2  =====
3  DDL Script: Create silver Tables
4  =====
5  Script Purpose:
6  This script creates tables in the 'silver' schema, dropping existing tables
7  if they already exist.
8  Run this script to re-define the DDL structure of 'silver' Tables
9  =====
10 */
11
12 -- 1. Customers Table
13 IF OBJECT_ID('silver.crm_customers', 'U') IS NOT NULL
14     DROP TABLE silver.crm_customers;
15 GO
16 CREATE TABLE silver.crm_customers(
17     customer_id          NVARCHAR(100),
18     customer_unique_id   NVARCHAR(100),
19     customer_zip_code_prefix INT,
20     customer_city         NVARCHAR(100),
21     customer_state       NVARCHAR(100),
22     dwh_create_date      DATETIME2 DEFAULT GETDATE()
23 );
24 GO
25
26 -- 2. Geolocation Table
27 IF OBJECT_ID('silver.crm_geolocation', 'U') IS NOT NULL
28     DROP TABLE silver.crm_geolocation;
29 GO
30 CREATE TABLE silver.crm_geolocation(
31     geolocation_zip_code_prefix INT,
32     geolocation_lat             DECIMAL(12, 9),
33     geolocation_lng             DECIMAL(12, 9),
34     geolocation_city            NVARCHAR(100),
35     geolocation_state           NVARCHAR(100),
36     dwh_create_date             DATETIME2 DEFAULT GETDATE()
37 );
38 GO
39
40 -- 3. Order Reviews Table
41 IF OBJECT_ID('silver.crm_order_reviews', 'U') IS NOT NULL
42     DROP TABLE silver.crm_order_reviews;
43 GO
44 CREATE TABLE silver.crm_order_reviews(
45     review_id          NVARCHAR(100),
46     order_id           NVARCHAR(100),
47     review_score        INT,
48     review_comment_title NVARCHAR(255),
49     review_comment_message NVARCHAR(MAX), -- Increased to handle long Portuguese reviews
50     review_creation_date DATETIME,
51     review_answers_timestamp DATETIME,
52     dwh_create_date      DATETIME2 DEFAULT GETDATE()
53 );
54 GO
55
56 -- 4. Order Items Table
57 IF OBJECT_ID('silver.erp_order_items', 'U') IS NOT NULL
58     DROP TABLE silver.erp_order_items;
59 GO
```

BUILD SILVER LAYER

CREATE SQL LOAD SCRIPTS

LOAD_SCRIPT_...rahlad (71) X DDL_SILVER.s...Prahlad (51)*

```
1  /*
2  =====
3  Stored Procedure: silver.usp_BulkLoadSilver
4  =====
5  Description:
6      Transforms and loads data from the Bronze layer to the Silver layer.
7      Standardizes data types, formatting, and casing across all 9 tables.
8
9  Usage:
10     EXEC silver.usp_BulkLoadSilver;
11  =====
12  */
13
14 CREATE OR ALTER PROCEDURE silver.usp_BulkLoadSilver AS
15 BEGIN
16     DECLARE @start_time DATETIME,
17             @end_time DATETIME,
18             @batch_start_time DATETIME,
19             @batch_end_time DATETIME;
20
21     BEGIN TRY
22         SET @batch_start_time = GETDATE();
23         PRINT '=====';
24         PRINT 'Loading Silver Layer';
25         PRINT '=====';
26
27         -----
28         -- 1. CRM DATA
29         -----
30         PRINT '-----';
31         PRINT 'Loading CRM Tables';
32         PRINT '-----';
33
34         -- Loading silver.crm_customers
35         SET @start_time = GETDATE();
36         PRINT '>> Truncating Table: silver.crm_customers';
37         TRUNCATE TABLE silver.crm_customers;
38         PRINT '>> Inserting Cleaned Data into: silver.crm_customers';
39
40         INSERT INTO silver.crm_customers (
41             customer_id,
42             customer_unique_id,
43             customer_zip_code_prefix,
44             customer_city,
45             customer_state,
46             dwh_create_date
47         )
48         SELECT
49             CAST(TRY_CAST(customer_id AS NVARCHAR(100)) AS NVARCHAR(100)) AS customer_id,
50             CAST(TRY_CAST(customer_unique_id AS NVARCHAR(100)) AS NVARCHAR(100)) AS customer_unique_id,
51             CAST(customer_zip_code_prefix AS INT) AS customer_zip_code_prefix,
52             ISNULL(UPPER(TRY_CAST(customer_city AS NVARCHAR(100))), 'UNKNOWN') AS customer_city,
53             ISNULL(UPPER(TRY_CAST(customer_state AS NVARCHAR(100))), 'NA') AS customer_state,
54             GETDATE() AS dwh_create_date
55         FROM (
56             SELECT *,
57                     ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY customer_id) AS rn
58             FROM bronze.crm_customers
59             WHERE customer_id IS NOT NULL
```

63 % No issues found

Ln: 1 Ch: 1 SPC CRLF

Connected. (1/1)

LAPTOP-VGO36MDC\SQL EXPRESS ... LAPTOP-VGO36MDC\Prahla... DataWarehouse 00:00:00 0 rows

Object Explorer

Connect

FileTables

External Tables

Graph Tables

bronze.crm_customers

bronze.crm_geolocation

bronze.crm_order_reviews

bronze.erp_order_items

bronze.erp_order_payments

bronze.erp_orders

bronze.erp_products

bronze.erp_sellers

bronze.product_category_name_translation

dbo.bronze_product_category_name_translation

silver.crm_customers

silver.crm_geolocation

silver.crm_order_reviews

silver.erp_order_items

silver.erp_order_payments

silver.erp_orders

silver.erp_products

silver.erp_sellers

silver.product_category_name_translation

Views

External Resources

Synonyms

Programmability

Stored Procedures

System Stored Procedures

bronze.usp_BulkLoadOlist

silver.usp_BulkLoadSilver

Functions

Database Triggers

Assemblies

Types

Rules

Defaults

LOAD_SCRIPT_...rahlad (71))

DDL_SILVER.s...Prahlad (51))*

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

```
/*
=====
Stored Procedure: silver.usp_BulkLoadSilver
=====
Description:
  Transforms and loads data from the Bronze layer to the Silver layer.
  Standardizes data types, formatting, and casing across all 9 tables.
Usage:
  EXEC silver.usp_BulkLoadSilver;
=====
*/

CREATE OR ALTER PROCEDURE silver.usp_BulkLoadSilver AS
BEGIN
  DECLARE @start_time DATETIME,
          @end_time DATETIME,
          @batch_start_time DATETIME,
          @batch_end_time DATETIME;

  BEGIN TRY
    SET @batch_start_time = GETDATE();
    PRINT '=====';
    PRINT 'Loading Silver Layer';
    PRINT '=====';

    -- 1. CRM DATA

    PRINT '-----';
    PRINT 'Loading CRM Tables';
    PRINT '-----';

    -- Loading silver.crm_customers
    SET @start_time = GETDATE();
    PRINT '>> Truncating Table: silver.crm_customers';
    TRUNCATE TABLE silver.crm_customers;
    PRINT '>> Inserting Cleaned Data into: silver.crm_customers';

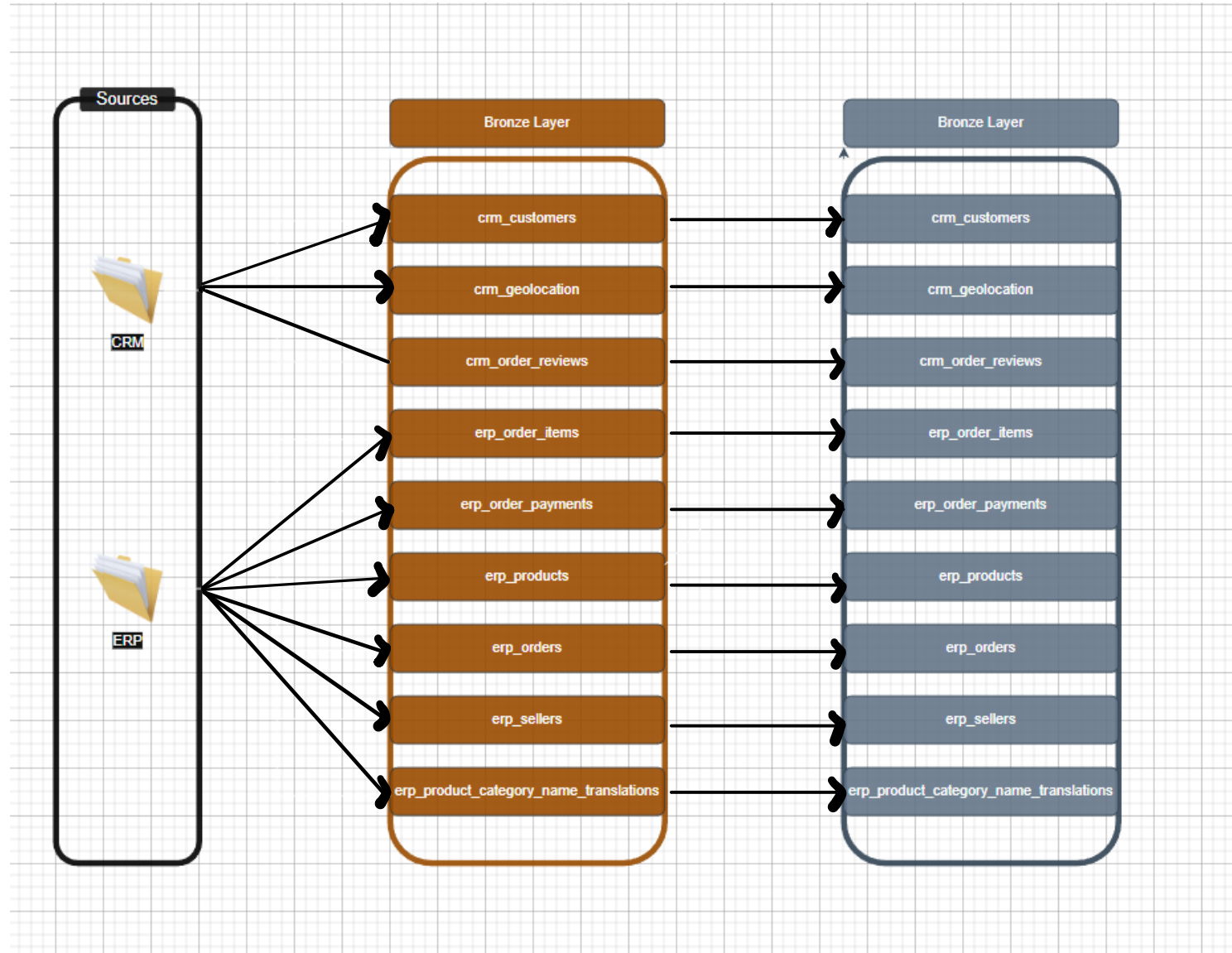
    INSERT INTO silver.crm_customers (
      customer_id,
      customer_unique_id,
      customer_zip_code_prefix,
      customer_city,
      customer_state,
      dwh_create_date
    )
    SELECT
      CAST(TRIM(customer_id) AS NVARCHAR(100)) AS customer_id,
      CAST(TRIM(customer_unique_id) AS NVARCHAR(100)) AS customer_unique_id,
      CAST(customer_zip_code_prefix AS INT) AS customer_zip_code_prefix,
      ISNULL(UPPER(TRIM(customer_city)), 'UNKNOWN') AS customer_city,
      ISNULL(UPPER(TRIM(customer_state)), 'NA') AS customer_state,
      GETDATE() AS dwh_create_date
    FROM (
      SELECT *,
             ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY customer_id) AS rn
      FROM bronze.crm_customers
      WHERE customer_id IS NOT NULL
    )
  
```

63 %

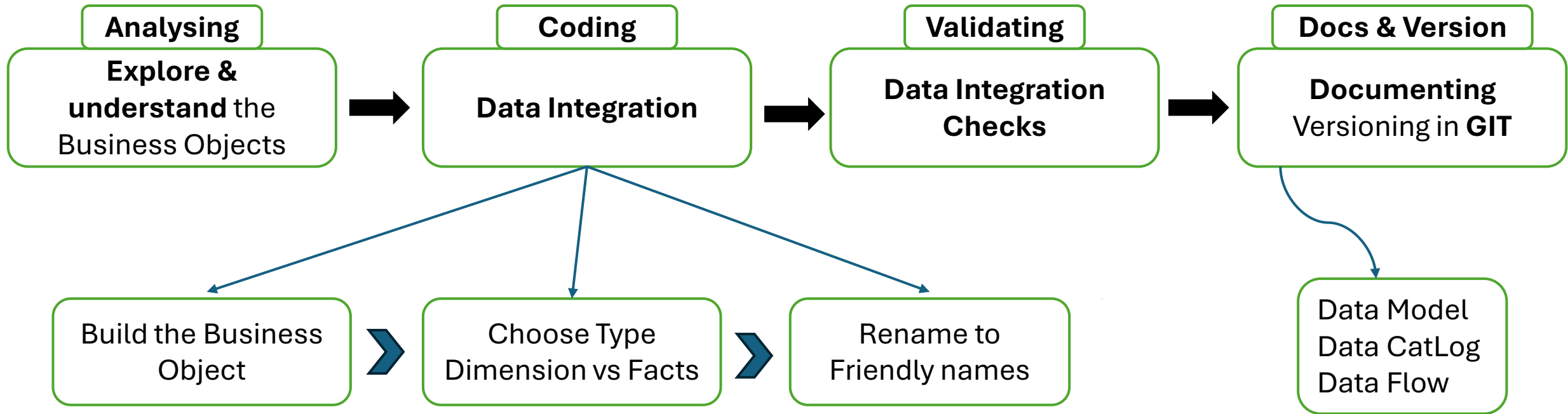
No issues found

Ln: 1 Ch: 1 SPC CRLF

DATA FLOW DIAGRAM

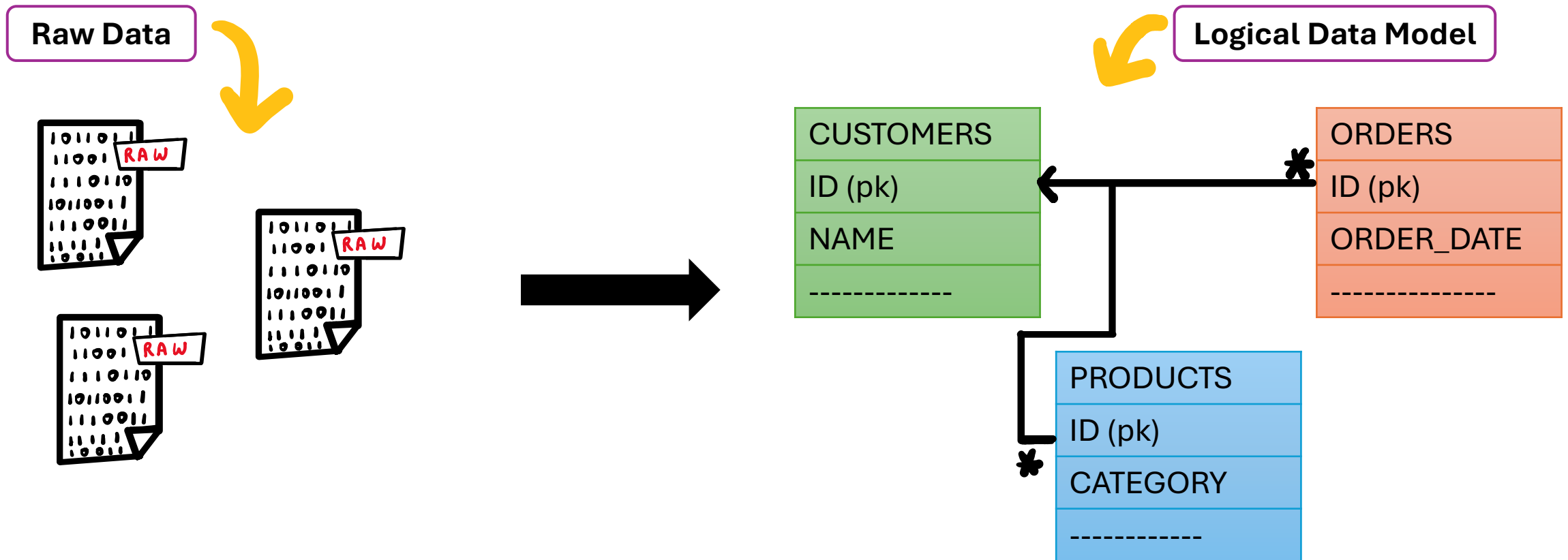


BUILDING GOLD LAYER



What Is Data Modelling

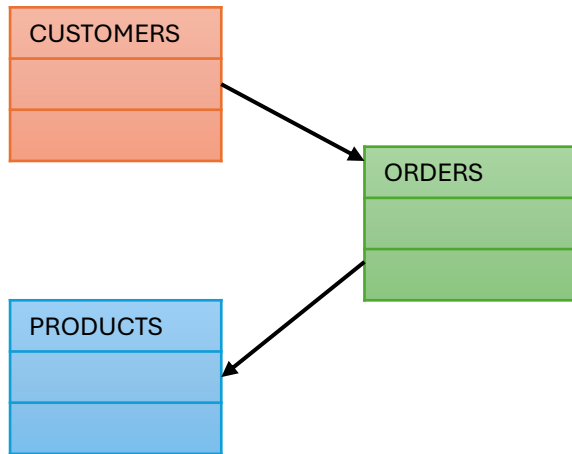
- Data modeling is the process of creating a visual blueprint for information systems, defining how data elements are stored, organized, and related to each other.
- It translates complex business requirements into structured database designs, ensuring data consistency, integrity, and improved performance for analytics, applications, and decision-making.



Levels of Data Modelling

Conceptual:

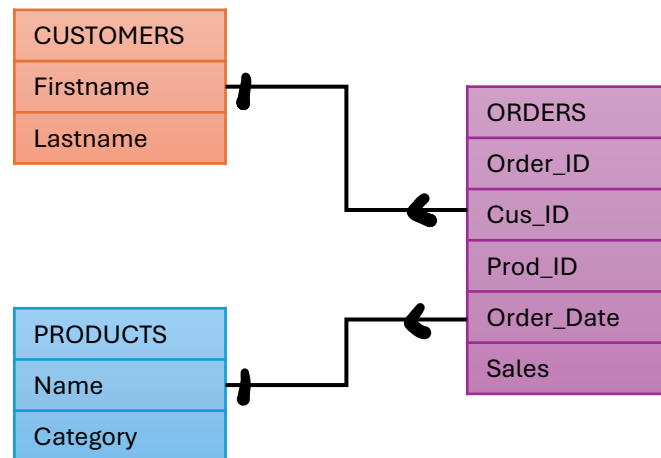
Defines high-level business concepts and scope.



BIG PICTURE

Logical:

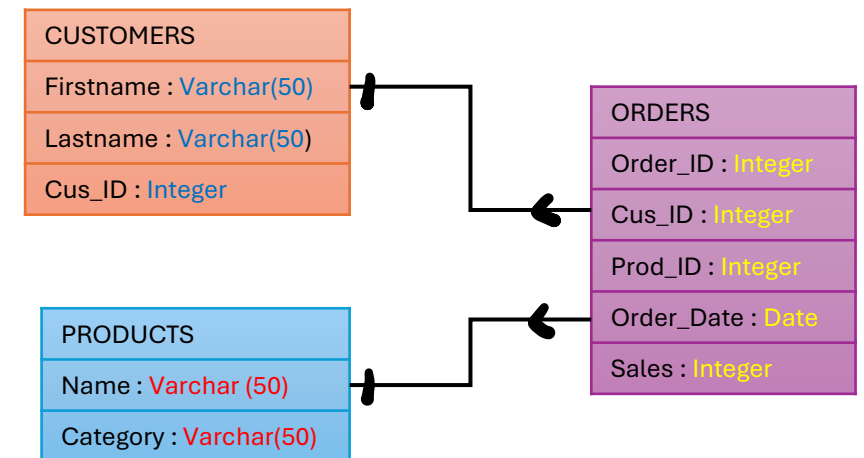
Maps out entity attributes and relationships without specific technology constraints.



BIG PRINT

Physical:

Defines specific table structures, columns, data types, and constraints for a particular database management system.



IMPLEMENTAION

STAR SCHEMA

VS

SNOWFLAKE SCHEMA

Star and snowflake schemas are foundational data modeling techniques used to organize data warehouses.

- ❑ The **star schema** uses a central fact table surrounded by denormalized dimensions for maximum query speed and simplicity.
- ❑ The **snowflake schema** further normalizes these dimensions into multiple related tables to reduce redundancy and save storage space.

Star Schema

Structure:

A single, central fact table connected directly to dimension tables. It is often visualized as a star.

Key Characteristic: Denormalized. Dimension tables contain redundant data (e.g., city, state, and country in one table).

Advantages:

- **Fast Query Performance:** Fewer tables mean fewer joins, which speeds up read performance.
- **Simplicity:** Easy to understand and navigate for business intelligence users.

Disadvantages:

- **Data Redundancy:** Higher storage usage due to redundant data.
- **Maintenance:** Data integrity must be managed carefully.

Snowflake Schema

Structure:

A central fact table connected to dimension tables, which are further normalized into sub-dimension tables, creating a snowflake-like structure.

Key Characteristic: Normalized. Data is structured to minimize redundancy (e.g., product, category, and sub-category are separated).

Advantages:

- **Storage Efficiency:** Reduced data redundancy saves disk space.
- **Data Integrity:** Normalized structures prevent update anomalies, making it easier to maintain data consistency.

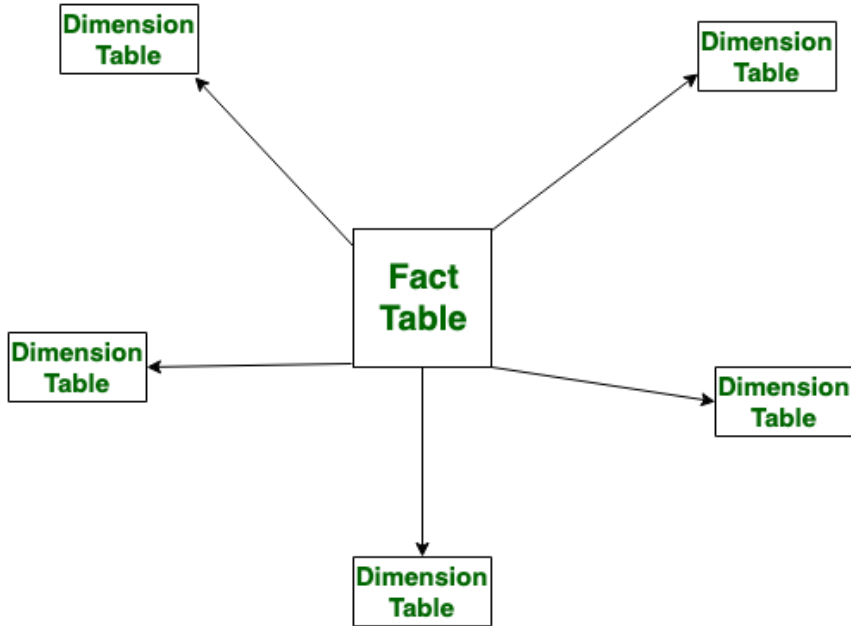
Disadvantages:

- **Slower Queries:** Increased joins required for queries can reduce performance.
- **Complexity:** More complex to design and query.

STAR SCHEMA

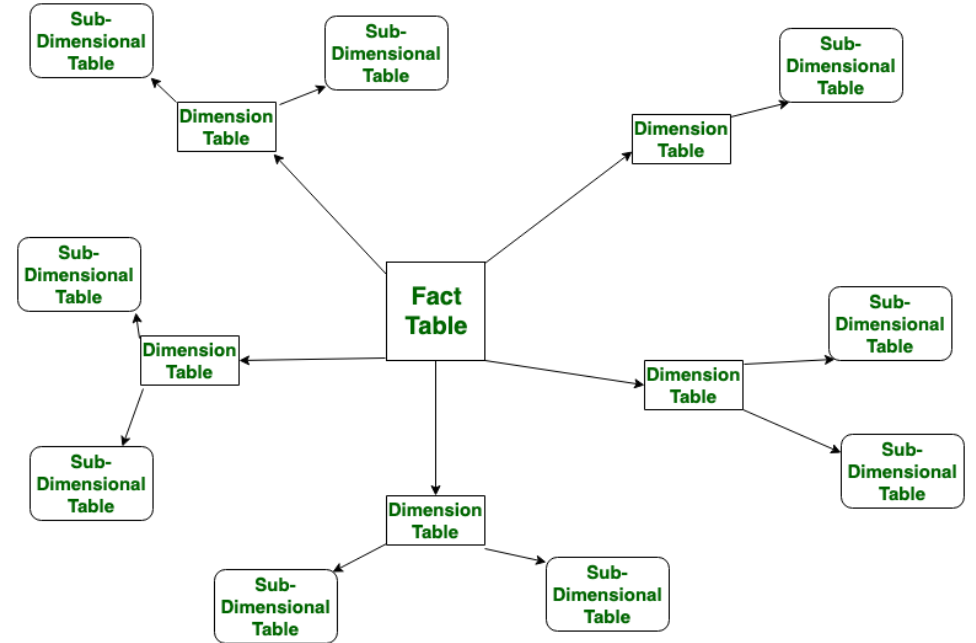
VS

SNOWFLAKE SCHEMA



SIMPLE & EASY

BIG DIMENSIONS



MORE COMPLEX

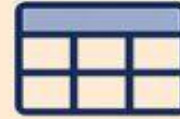
LARGE DATASETS

Difference Between Dimension and Fact Table



DIMENSION TABLE

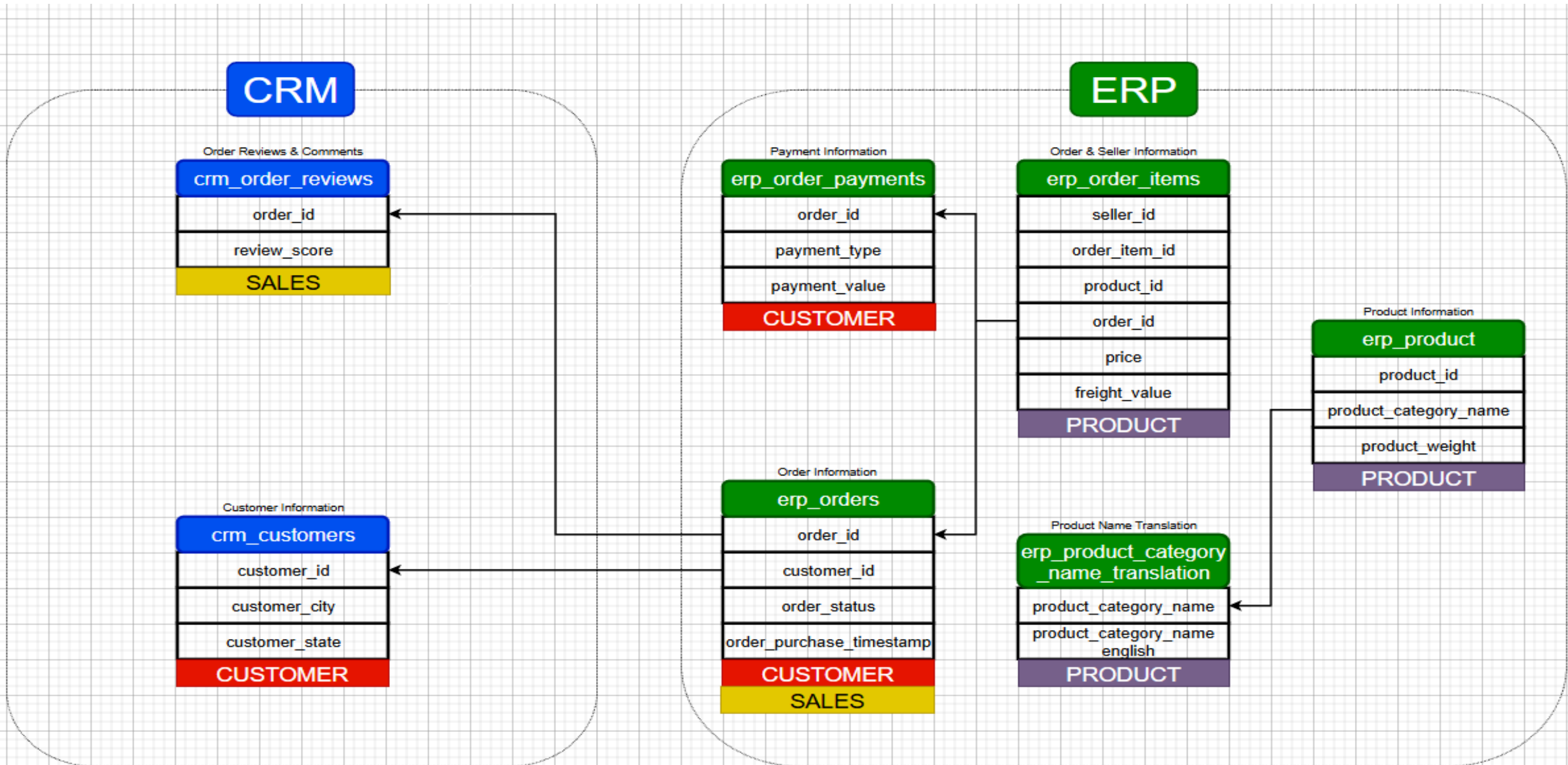
Purpose	Describes business entities
Data Type	Textual / Descriptive
Primary Key	Surrogate / Natural Key
Examples	Customer, Product



FACT TABLE

Purpose	Stores business events
Data Type	Numeric / Quantitative
Primary Key	Composite / Foreign Keys
Examples	Sales, Orders, Revenue

INTEGRATION MODEL (how tables are related)



EXPLORING THE DOMAIN-DRIVEN TABLE RELATIONSHIP & BUSINESS LOGIC & IMPLITATION ARCHITECTURE

The **Gold Layer Integration Model** serves as the structural blueprint for transforming raw, siloed source(**Silver layer**) data into a unified, business-ready **Star Schema**. This model establishes the logical connections required to bridge the gap between **CRM** and **ERP** systems to create a "Single Source of Truth."

1. Domain-Driven Table Relationships :-

The model categorizes tables into distinct business domains, represented by the color-coded tags at the bottom of each table:

A. Customer Domain (Red Tag):

- Tables Related:** `crm_customers` (CRM), `erp_orders` (ERP), and `erp_order_payments` (ERP).
- Primary Keys/Foreign Keys:** These tables are linked via `customer_id` and `order_id`.
- Business Logic:** This creates a **Customer 360 View**. By connecting demographic data from the CRM with transactional behavior from the ERP, the system can calculate **Customer Lifetime Value (CLV)**—understanding which geographic regions or customer profiles generate the highest revenue over time.

B. Product Domain (Purple Tag):

- Tables Related:** `erp_product`, `erp_product_category_name_translation`, and `erp_order_items`.
- Primary Keys/Foreign Keys:** These are linked via `product_id` and `product_category_name`.
- Business Logic:** This establishes a **Product Master Catalog**. The specific link to the translation table fulfills the business objective of providing **Internationalized Reporting**. It converts Portuguese category names into English, ensuring the Gold Layer is ready for global business stakeholders.

2. Strategic Business Logic Discovered :-

The integration model uncovers analytical logic previously hidden in isolated files:

- Sentiment-to-Revenue Bridge:** The connection between `erp_orders` and `crm_order_reviews` via `order_id` is a high-value link. It allows the business to determine if operational factors—such as high freight costs or delivery delays found in the ERP—are the direct cause of poor customer review scores in the CRM.
- Indirect Chain of Logic:** The model identifies that `crm_customers` and `erp_order_items` are only related **indirectly** through the `erp_orders` table. The Gold Layer implementation automates this "multi-hop" join, providing a simplified, denormalized view for end-users.

3. Implementation Architecture

The translation of this integration model into SQL DDL applies two professional engineering standards:

- 1.Surrogate Key Framework:** Using `ROW_NUMBER()` to create keys like `customer_key` and `product_key` ensures the warehouse remains stable even if source-system IDs change.
- 2.Star Schema Performance:** Grouping these related tables into a central Fact (`fact_sales`) and surrounding Dimensions (`dim_customers`, `dim_products`) ensures the database is optimized for complex analytical queries, such as churn prediction and RFM analysis.

BUILD GOLD LAYER

CREATE DIMENSION & FACT TABLE

Object Explorer

Connect

- FileTables
- External Tables
- Graph Tables
- bronze.crm_customers
- bronze.crm_geolocation
- bronze.crm_order_reviews
- bronze.erp_order_items
- bronze.erp_order_payments
- bronze.erp_orders
- bronze.erp_products
- bronze.erp_sellers
- bronze.product_category_name_translat
- dbo.bronze_product_category_name_tra
- silver.crm_customers
- silver.crm_geolocation
- silver.crm_order_reviews
- silver.erp_order_items
- silver.erp_order_payments
- silver.erp_orders
- silver.erp_products
- silver.erp_sellers
- silver.product_category_name_translatio

- Views
- External Resources
- Synonyms
- Programmability
 - Stored Procedures
 - System Stored Procedures
 - bronze.usp_BulkLoadOlist
 - silver.usp_BulkLoadSilver
- Functions
- Database Triggers
- Assemblies
- Types
- Rules
- Defaults

DDL_GOLD.sql...rahlad (56)

LOAD_SCRIPT_S...Prahla (71)

DDL_SILVER.s...Prahla (51)*

```
1  /*
2  =====
3  DDL Script: Create Gold Views
4  =====
5  Script Purpose:
6  This script creates views for the Gold layer in the data warehouse.
7  The Gold layer represents the final dimension and fact tables (Star Schema).
8
9  Each view performs transformations and combines data from the Silver layer
10 to produce a clean, enriched, and business-ready dataset.
11
12 Usage:
13 - These views can be queried directly for analytics and reporting.
14 =====
15 */
16
17 -- =====
18 -- 1. Create Dimension: gold.dim_customers
19 -- =====
20 CREATE OR ALTER VIEW gold.dim_customers AS
21 SELECT
22     ROW_NUMBER() OVER (ORDER BY customer_id) AS customer_key,
23     customer_id AS customer_id,
24     customer_city AS city,
25     customer_state AS state
26 FROM silver.crm_customers;
27 GO
28
29 -- =====
30 -- 2. Create Dimension: gold.dim_products
31 -- =====
32 CREATE OR ALTER VIEW gold.dim_products AS
33 SELECT
34     ROW_NUMBER() OVER (ORDER BY p.product_id) AS product_key,
35     p.product_id AS product_id,
36     p.product_weight_g AS weight_grans,
37     ISNULL(t.product_category_name_english, 'Other') AS product_category
38 FROM silver.erp_products p
39 LEFT JOIN silver.product_category_name_translation t
40     ON p.product_category_name = t.product_category_name;
41 GO
42
43 -- =====
44 -- 3. Create Fact Table: gold.fact_sales
45 -- =====
46 CREATE OR ALTER VIEW gold.fact_sales AS
47 SELECT
48     c.customer_key,
49     p.product_key,
50     o.order_id AS order_id,
51     o.order_purchase_timestamp AS order_date,
52     -- Aggregated Business Metrics
53     SUM(oi.price) AS item_revenue,
54     SUM(oi.freight_value) AS shipping_cost,
55     SUM(oi.price + oi.freight_value) AS total_sales_amount,
56     COUNT(*) AS items_quantity
57 FROM silver.erp_orders o
58 INNER JOIN silver.erp_order_items oi
59     ON o.order_id = oi.order_id
60 LEFT JOIN gold.dim_customers c
61     ON o.customer_id = c.customer_id
62 LEFT JOIN gold.dim_products p
63     ON oi.product_id = p.product_id
64 GROUP BY
65     c.customer_key,
66     p.product_key,
67     o.order_id,
68     o.order_purchase_timestamp;
69 GO
```

52 %

No issues found

Ln: 1 Ch: 1 SPC CRLF

Connected. (1/1)

LAPTOP-VGO36MDC\SQLSERVER ...

LAPTOP-VGO36MDC\Prahla...

DataWarehouse

00:00:00

0 rows

- FileTables
- External Tables
- Graph Tables
- bronze.crm_customers
- bronze.crm_geolocation
- bronze.crm_order_reviews
- bronze.erp_order_items
- bronze.erp_order_payments
- bronze.erp_orders
- bronze.erp_products
- bronze.erp_sellers
- bronze.product_category_name_translation
- dbo.bronze_product_category_name_translation
- silver.crm_customers
- silver.crm_geolocation
- silver.crm_order_reviews
- silver.erp_order_items
- silver.erp_order_payments
- silver.erp_orders
- silver.erp_products
- silver.erp_sellers
- silver.product_category_name_translation
- Views
 - System Views
 - gold.dim_customers
 - gold.dim_products
 - gold.fact_sales
- External Resources
- Synonyms
- Programmability
 - Stored Procedures
 - System Stored Procedures
 - bronze.usp_BulkLoadOlist
 - silver.usp_BulkLoadSilver
 - Functions
 - Database Triggers

```
1  /*
2  =====
3  DDL Script: Create Gold Views
4  =====
5  Script Purpose:
6      This script creates views for the Gold layer in the data warehouse.
7      The Gold layer represents the final dimension and fact tables (Star Schema).
8
9      Each view performs transformations and combines data from the Silver layer
10     to produce a clean, enriched, and business-ready dataset.
11
12 Usage:
13     - These views can be queried directly for analytics and reporting.
14 =====
15 */
16
17 -- =====
18 -- 1. Create Dimension: gold.dim_customers
19 -- =====
20 CREATE OR ALTER VIEW gold.dim_customers AS
21 SELECT
22     ROW_NUMBER() OVER (ORDER BY customer_id) AS customer_key,
23     customer_id AS customer_id,
24     customer_city AS city,
25     customer_state AS state
26 FROM silver.crm_customers;
27 GO
28
29 -- =====
30 -- 2. Create Dimension: gold.dim_products
31 -- =====
32 CREATE OR ALTER VIEW gold.dim_products AS
33 SELECT
34     ROW_NUMBER() OVER (ORDER BY p.product_id) AS product_key,
35     p.product_id AS product_id,
36     p.product_weight_g AS weight_grans,
37     ISNULL(t.product_category_name_english, 'Other') AS product_category
38 FROM silver.erp_products p
39 LEFT JOIN silver.product_category_name_translation t
40     ON p.product_category_name = t.product_category_name;
41 GO
42
43 -- =====
44 -- 3. Create Fact Table: gold.fact_sales
45 -- =====
46 CREATE OR ALTER VIEW gold.fact_sales AS
47 SELECT
48     c.customer_key,
49     p.product_key,
50     o.order_id AS order_id,
51     o.order_purchase_timestamp AS order_date,
52     -- Aggregated Business Metrics
53     SUM(oi.price) AS item_revenue,
54     SUM(oi.freight_value) AS shipping_cost,
55     SUM(oi.price + oi.freight_value) AS total_sales_amount,
56     COUNT(*) AS items_quantity
57 FROM silver.erp_orders o
58 INNER JOIN silver.erp_order_items oi
59     ON o.order_id = oi.order_id
60 LEFT JOIN gold.dim_customers c
61     ON o.customer_id = c.customer_id
62 LEFT JOIN gold.dim_products p
63     ON oi.product_id = p.product_id
64 GROUP BY
65     c.customer_key,
66     p.product_key,
67     o.order_id,
68     o.order_purchase_timestamp;
69 GO
```

GOLD LAYER MASTER VALIDATION

Validating_g...Prahla (54) x DDL_GOLD.sql...Prahla (56) LOAD_SCRIPT_S...Prahla (71) DDL_SILVER.s...Prahla (51)*

```
1  /* =====
2  GOLD LAYER MASTER VALIDATION: Olist Data Warehouse
3  =====
4  Check 1: Orphaned Keys (Referential Integrity)
5  Check 2: Financial Balance (Gold vs Silver Totals)
6  Check 3: Logical Accuracy (Count of Categorized Products)
7  =====
8  */
9
10 SELECT
11     'Referential Integrity' AS check_type,
12     'Orphaned Product Keys' AS metric,
13     COUNT(*) AS value
14 FROM gold.fact_sales
15 WHERE product_key IS NULL
16
17 UNION ALL
18
19 SELECT
20     'Referential Integrity',
21     'Orphaned Customer Keys',
22     COUNT(*)
23 FROM gold.fact_sales
24 WHERE customer_key IS NULL
25
26 UNION ALL
27
28 SELECT
29     'Financial Balance',
30     'Total Revenue Variance (ERP vs Gold)',
31     (SELECT SUM(price + freight_value) FROM silver.erp_order_items) -
32     (SELECT SUM(total_sales_amount) FROM gold.fact_sales)
33
34 UNION ALL
35
36 SELECT
37     'Logical Completeness',
38     'Unmapped Product Categories (Total)',
39     COUNT(*)
40 FROM gold.dim_products
41 WHERE product_category = 'Other';
```

63 % No issues found

Ln: 41 Ch: 33 SPC CRLF

Results Messages

	check_type	metric	value
1	Referential Integrity	Orphaned Product Keys	0.00
2	Referential Integrity	Orphaned Customer Keys	0.00
3	Financial Balance	Total Revenue Variance (ERP vs Gold)	0.00
4	Logical Completeness	Unmapped Product Categories (Total)	623.00

Query executed successfully.

LAPTOP-VGO36MDC\SQLEXPRESS ... LAPTOP-VGO36MDC\Prahla... DataWarehouse 00:00:01 4 rows

BUILD GOLD LAYER

BUILD THE STAR SCHEMA MODEL



1. Core Architectural Components (The Star Schema) :

The diagram illustrates how you successfully decouple descriptive context from quantitative measurements:

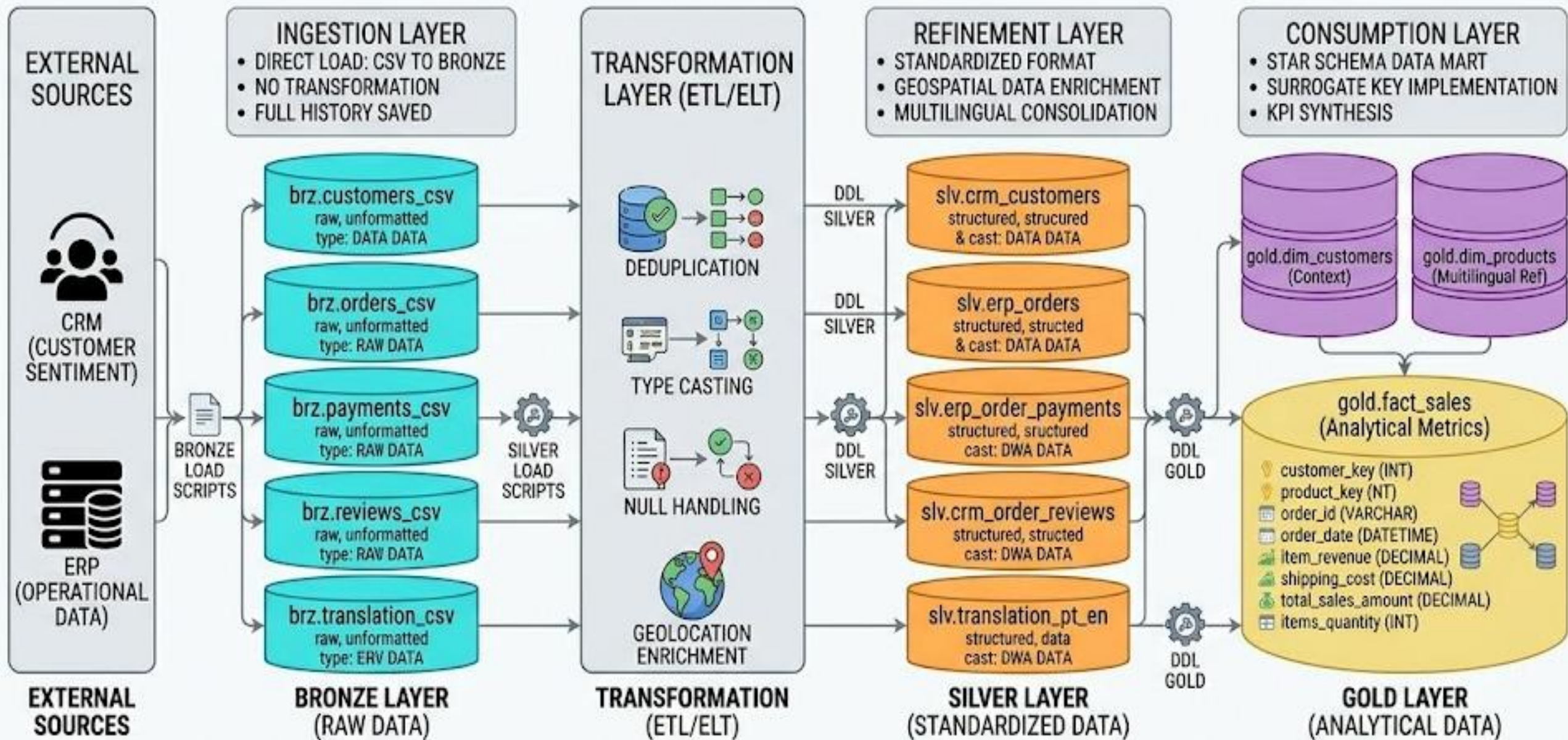
- **The Central Hub (Fact Sales):** The yellow entity represents `gold.fact_sales`, containing transactional data such as `total_sales_amount` and `items_quantity`. This is the highest level of aggregation for sales metrics.
- **Contextual Dimensions (Customers & Products):** The blue and purple surrounding tables, `gold.dim_customers` and `gold.dim_products`, provide the "Who" and "What" of every transaction. This **denormalized design** reduces join complexity for end-user queries, a critical requirement for your final submission.

2. Relationship Mapping via Surrogate Keys :

The connection lines between the tables highlight several professional data engineering practices implemented in your script:

- **Stable Schema Logic:** The dimension tables use **Surrogate Keys** (e.g., `customer_key`) generated by `ROW_NUMBER()`. This architecture ensures your fact table remains accurate even if the underlying `customer_id` changes in the source system.
- **Referential Integrity:** The logic successfully bridges **CRM (Customers)** and **ERP (Order Items)** silos. As verified in your previous validation scripts, this design maintains perfect financial balance and referential integrity.

DATA FLOW DIAGRAM: OLIST DATA WAREHOUSE ARCHITECTURE



VISUALIZING THE BRONZE TO GOLD ETL PROCESS WITH BUSINESS LOGIC INTEGRATION

THANK YOU